

CVM++ Project and Build Guide



::: infobox{What this guide covers} Everything you need to **clone, build, and understand** CVM++: what each folder does, how the project was built step by step, and what every technical term means. Pair with the **Architecture and Workflow** PDF for runtime internals. :::

What is CVM++?

CVM++ is a **scripting language + toolchain** in C++17. You write `.cvm` files; one program (`cvmp`) compiles and runs them.

Stage	Input	Output
Lexer	Characters	Tokens
Parser	Tokens	AST (tree)
Compiler	AST	Bytecode bytes
VM	Bytecode	Printed results

You are **not** using LLVM, Python, or an external interpreter library — the full pipeline lives in this repository.

Prerequisites

Tool	Purpose
C++17 compiler (GCC 9+, Clang 10+, MSVC 2019+)	Build <code>cvmp</code>
Make	<code>make</code> , <code>make verify</code>
CMake 3.16+ (optional)	Alternative build
Git	Clone / publish

To **regenerate these PDFs** after editing Markdown: `./scripts/build-docs.sh` (requires Pandoc + LaTeX).

Quick start

```
git clone https://github.com/Dragon9-9/cvmp.git
cd cvmp
make
./build/cvmp examples/hello.cvm
./build/cvmp -d examples/functions.cvm
make verify
```

Flag	Effect
<code>-d</code>	Show tokens, AST, bytecode
<code>-q</code>	Quiet (program output only)
<code>-c</code>	Compile only (no VM)

Exit codes: 0 success · 1 compile error · 2 runtime error

Repository map

```

cvmpp/
├─ include/cvm++/    Headers (lexer, parser, ast, compiler, vm, ...)
├─ src/              Implementations
├─ examples/         Sample .cvm scripts
├─ docs/pdf/         These guides (PDF)
├─ scripts/          verify.sh, build-docs.sh
├─ Makefile           Primary build
└─ .github/workflows/ CI on push

```

How the project was built (recommended order)

:: infobox{Step 1 — Foundation} source_loc, diagnostic, token, lexer, ui, minimal main that prints tokens. ::

:: infobox{Step 2 — Parser & AST} ast.hpp, recursive-descent parser, ast_print for debugging. ::

:: infobox{Step 3 — Bytecode} opcode, bytecode writer + disassembler, compiler with jump patching. ::

:: infobox{Step 4 — Virtual machine} value (std::variant), vm stack loop, compile_frontend wiring. ::

:: infobox{Step 5 — Product} REPL, file runner, functions (fn/return/call), examples, CI, docs. ::

Language reference

```

let x = 10;
fn add(a, b) { return a + b; }
if (x < 100) { print add(x, 1); }
while (x > 0) { x = x - 1; }
let n = input;
print n;

```

Feature	Syntax
Types	integers, true / false
Variables	let x = ...; and x = ...;
Functions	fn name(a) { return ...; } · name(1, 2)
Control	if / else, while
Operators	+ - * / == != < > <= >=
I/O	print expr; · let x = input;
Comments	// ...

Not included: strings, floats, arrays, for, modules.

Example programs

Script	Shows
hello.cvm	Basics
functions.cvm	Recursion
factorial.cvm	while
comparisons.cvm	!= <= >=
div_by_zero.cvm	Runtime error (exit 2)

Glossary

Term	Meaning
Token	Classified source chunk (e.g. let, 42, ==)
AST	Tree of program structure after parsing
Bytecode	std::vector<uint8_t> VM instructions
Opcode	One-byte instruction tag (Add, Call, ...)
Stack	Where expression values live during execution
Call frame	Saved return address + local slots for a function
IP	Instruction pointer — index into bytecode
Name pool	Interned variable names in the bytecode chunk
Diagnostic	Error with phase, line/column, message, hint
VmSession	REPL map that keeps globals between lines
Front end	Lexer + parser + compiler together

Troubleshooting

Problem	Fix
cvmp not found	Run make; use ./build/cvmp
Verify fails	./build/cvmp -d examples/<file>.cvm
PDF build fails	Install Pandoc + MacTeX/TeX Live